

# Самостоятельная работа №3: Аутентификация и управление доступом с помощью токенов JWT

---

## 1. Цель самостоятельной работы

---

Цель данной самостоятельной работы состоит в изучении и практической реализации механизма аутентификации и управления доступом с использованием токенов JWT (JSON Web Tokens) на языке программирования Python. В ходе выполнения работы студенты научатся создавать и верифицировать JWT, а также применять их для защиты RESTful API.

## 2. Подробное техническое задание к самостоятельной работе

---

1. Установить необходимые библиотеки `pyjwt` и `Flask` для работы с JWT и создания веб-приложений в Python.
2. Создать простое веб-приложение на Flask, которое будет реализовывать следующие функции:
  - Регистрация пользователя с сохранением учетных данных в базе данных (можно использовать SQLite для простоты).
  - Аутентификация пользователя с выдачей JWT при успешном логине.
  - Защита эндпоинтов приложения с использованием JWT для управления доступом.
  - Обновление токенов и реализация механизма "refresh token".
3. Реализовать обработку ошибок и исключительных ситуаций.
4. Протестировать реализацию на нескольких примерах, убедившись в правильности работы аутентификации и управления доступом.

## 3. Список рекомендаций по использованию библиотек, структур данных, паттернов проектирования и так далее

---

### Библиотеки

- `pyjwt`: Библиотека для работы с JSON Web Tokens в Python. Установите её с помощью команды:

```
pip install pyjwt
```

- `Flask`: Легковесный фреймворк для создания веб-приложений. Установите его с помощью команды:

```
pip install Flask
```

- `Flask-SQLAlchemy`: Библиотека для работы с базами данных в Flask. Установите её с помощью команды:

```
pip install Flask-SQLAlchemy
```

## Структуры данных

- Используйте словари для хранения данных пользователя, токенов и других параметров аутентификации.
- Используйте базу данных SQLite для хранения учетных данных пользователей.

## Паттерны проектирования

- **Функциональный подход:** Разделите ваш код на небольшие функции, каждая из которых выполняет конкретную задачу (например, регистрация, аутентификация, проверка токенов).
- **Обработка исключений:** Используйте блоки `try-except` для обработки возможных ошибок при работе с JWT и базой данных.
- **Blueprints в Flask:** Используйте Blueprints для организации кода вашего приложения Flask.

## 4. Формат отчета по самостоятельной работе

---

Отчет по самостоятельной работе должен включать следующие разделы:

### Введение

- Краткое описание цели и задач самостоятельной работы.

### Теоретическая часть

- Описание механизма аутентификации с использованием JWT.
- Принципы работы JWT, структура токена и процесс верификации.

### Практическая часть

- Описание используемых инструментов и библиотек.
- Исходный код программы с комментариями.
- Скриншоты или вывод результатов тестирования программы.

### Заключение

- Выводы по результатам выполнения самостоятельной работы.
- Описание возможных улучшений и доработок.

### Приложения

- Дополнительные материалы, если таковые имеются (например, тестовые данные, дополнительный код).

---

## Пример реализации в Python

```
from flask import Flask, request, jsonify
from flask_sqlalchemy import SQLAlchemy
```

```

from werkzeug.security import generate_password_hash, check_password_hash
import jwt
import datetime

app = Flask(__name__)
app.config['SECRET_KEY'] = 'your_secret_key'
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///users.db'
db = SQLAlchemy(app)

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(50), unique=True, nullable=False)
    password = db.Column(db.String(80), nullable=False)

def token_required(f):
    def decorated(*args, **kwargs):
        token = request.headers.get('x-access-tokens')
        if not token:
            return jsonify({'message': 'Token is missing!'}), 403
        try:
            data = jwt.decode(token, app.config['SECRET_KEY'], algorithms=
["HS256"])
            current_user =
User.query.filter_by(username=data['username']).first()
        except:
            return jsonify({'message': 'Token is invalid!'}), 403
        return f(current_user, *args, **kwargs)
    return decorated

@app.route('/register', methods=['POST'])
def register():
    data = request.get_json()
    hashed_password = generate_password_hash(data['password'], method='sha256')
    new_user = User(username=data['username'], password=hashed_password)
    db.session.add(new_user)
    db.session.commit()
    return jsonify({'message': 'New user registered!'})

@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    user = User.query.filter_by(username=data['username']).first()
    if not user or not check_password_hash(user.password, data['password']):
        return jsonify({'message': 'Invalid credentials!'}), 401
    token = jwt.encode({'username': user.username, 'exp':
datetime.datetime.utcnow() + datetime.timedelta(minutes=30)},
app.config['SECRET_KEY'])
    return jsonify({'token': token})

@app.route('/protected', methods=['GET'])
@token_required
def protected_route(current_user):
    return jsonify({'message': f'Hello, {current_user.username}!'})

if __name__ == '__main__':
    db.create_all()

```

```
app.run(debug=True)
```

Соблюдайте структуру отчета и приводите результаты тестирования, чтобы продемонстрировать правильность работы вашей реализации.